

Python Master Practice Curriculum

An expanded, unified workbook compiling fundamental core scripts alongside professional-tier algorithmic engineering paradigms.

Total Problems: 60 Curated Production Tasks

Difficulty Scaling: Linear Matrix (Basic → Intermediate → Advanced)

Expanded Paradigms Added: Functional Scope, Object Blueprints, Exception Traps, I/O Files

Verification Suite: Independent Edge-Case Sandbox Inputs & Outputs

1. Variables, Data Types & Operators

Mastering variable declaration, data type mutability, numeric operations, and logical/bitwise evaluation.

Q1. Temperature Unit Converter

BASIC

Write a Python script that accepts a temperature in Celsius and converts it to both Fahrenheit and Kelvin. Ensure the script prints the results in a clean, human-readable statement.

Sample Input: Celsius: 25

Expected Output: 25°C is equivalent to 77.0°F and 298.15K

Q2. Arithmetic Operator Evaluator

BASIC

Create a script that requests two integers from the user and calculates their sum, difference, product, exact quotient, floor quotient, and remainder using standard python operators.

Sample Input: Num1 = 15, Num2 = 4

Expected Output: Sum: 19, Diff: 11, Prod: 60, Div: 3.75, Floor Div: 3, Modulo: 3

Q3. Compound Interest Financial Engine

INTERMEDIATE

Write a program to calculate the compound interest of an investment. The script should request the principal amount (P), annual interest rate (r), time in years (t), and number of times interest is compounded per year (n). Output the final accrued amount rounded to exactly two decimal places.

Sample Input: P = 5000, r = 5% (0.05), t = 3, n = 4

Expected Output: Total Accrued Amount: \$5803.77

Q4. Seconds to Digital Time Formatter

INTERMEDIATE

Given a large integer representing a duration in total seconds, convert it into standard digital format tracking Days, Hours, Minutes, and Seconds (DD:HH:MM:SS).

Sample Input: Seconds = 90061

Expected Output: Formatted Time: 01 days, 01 hours, 01 minutes, 01 seconds

Q5. Bitwise Power-of-Two Verifier

ADVANCED

Write a highly optimized program that checks whether a given positive integer is a perfect power of two. Constraint: You must achieve this using only bitwise operators (&, |, ^, ~, <<, >>) without any loops, math libraries, or conditional statements for the computation logic.

Sample Input: Number = 16 | Number = 18

Expected Output: True | False

Q6. Pure Mathematical Base-10 to Binary Converter

ADVANCED

Read a positive integer in base-10 and convert it into its binary (base-2) equivalent string. Constraint: You are forbidden from using the built-in bin() function, formatting expressions like f'{num:b}', or pre-existing string casting methods. Use arithmetic, modulo, and floor division operators manually.

Sample Input: Integer = 13

Expected Output: Binary String: '1101'

2. Strings, Indexing & Slicing

Manipulating textual data, executing advanced string slicing, and applying string object methods.

Q7. Normalized Palindrome Checker

BASIC

Write a program to determine if a given string is a palindrome. The program must be case-insensitive and ignore all spaces and punctuation marks.

Sample Input: String = 'A man a plan a canal Panama'

Expected Output: Result: True (It is a palindrome)

Q8. Boundary Character Extractor

BASIC

Extract the first 3 characters and the last 3 characters of an input string and concatenate them together. If the input string contains fewer than 6 characters, return a message saying 'String too short'.

Sample Input: String = 'PythonProgramming'

Expected Output: Extracted String: 'Pyting'

Q9. Secure Credit Card Masker

INTERMEDIATE

Implement a privacy compliance utility that takes a 16-digit credit card number string. It must replace the first 12 characters with asterisks (*) and format the output so there is a hyphen breaking up every group of 4 digits.

Sample Input: Card = '1234567812345678'

Expected Output: Masked Format: '****-****-****-5678'

Q10. Sentence-Level Word Reverser

INTERMEDIATE

Given an input sentence string, reverse the absolute order of the words while keeping the internal characters of each individual word perfectly intact. Ensure trailing/leading spaces are handled gracefully.

Sample Input: Sentence = 'Data Science with Python is powerful'

Expected Output: Reversed Sentence: 'powerful is Python with Data Science'

Q11. Run-Length String Compression Engine

ADVANCED

Implement a basic string compression algorithm using the counts of repeated consecutive characters. For example, 'aabcccccaaa' becomes 'a2b1c5a3'. Constraint: If the 'compressed' string would not become strictly shorter than the original string, your program must return the original string.

```
Sample Input:   String = 'aabcccccaaa' | String = 'abcd'
Expected Output: Result: 'a2b1c5a3'      | Result: 'abcd'
```

Q12. Substring Frequency Analysis Engine

ADVANCED

Write a program that counts how many times a specific substring occurs inside a parent body of text. Constraint: Do not use the built-in string method `.count()`. You must build your own traversal window using manual string slicing and loops.

```
Sample Input:   Text = 'ABCD CDC', Substring = 'CDC'
Expected Output: Total Occurrences: 2 (Note: Overlapping instances must be counted)
```

3. Lists & Tuples

Working with ordered collections, sequence mutability vs immutability, and indexing matrices.

Q13. Manual Sequence Statistics Engine

BASIC

Given a list of numeric values, compute the maximum element, minimum element, and the mathematical average. Constraint: You are forbidden from using the built-in functions `max()`, `min()`, or `sum()`. Traverse the sequence manually.

```
Sample Input:   List = [23, 45, 12, 56, 89, 34]
Expected Output: Max: 89, Min: 12, Average: 43.83
```

Q14. Order-Preserving Duplicate Purger

BASIC

Write a program that cleans a list by removing all duplicate entries. Constraint: You must preserve the exact chronological sequence order of the elements as they appeared originally. Do not rely on casting to a standard unordered set.

Sample Input: List = [1, 3, 2, 1, 4, 3, 2, 5]

Expected Output: Cleaned List: [1, 3, 2, 4, 5]

Q15. Cyclic List Shift Engine

INTERMEDIATE

Design a routine to perform a left cyclic rotation on a list by an integer factor of k. This means elements are shifted leftward, wrapping around to the tail. Handle large inputs where k is vastly greater than the list's total length.

Sample Input: List = [10, 20, 30, 40, 50], k = 2

Expected Output: Rotated List: [30, 40, 50, 10, 20]

Q16. Immutable Record Sort Optimizer

INTERMEDIATE

You are given a tuple containing student records, where each record is itself a tuple of (Name, Score). Write a program to sort this collection in descending order based exclusively on their scores, producing a sorted tuple of tuples.

Sample Input: Records = (('Alice', 88), ('Bob', 95), ('Charlie', 78))

Expected Output: Sorted: (('Bob', 95), ('Alice', 88), ('Charlie', 78))

Q17. Two-Sum Unique Pair Extractor

ADVANCED

Given an unsorted list of integers and a target total integer, isolate all completely unique pairs of numbers within the list whose summation precisely matches the target value. Avoid duplicate pairs in your final list.

Sample Input: List = [2, 4, 3, 5, 7, 8, 9], Target = 7

Expected Output: Unique Pairs: [(2, 5), (3, 4)]

Q18. Pure List-Based Matrix Transposition

ADVANCED

Write an algorithm that takes an M x N 2D matrix (represented as nested lists) and computes its mathematical transpose (turning rows into columns and vice-versa) resulting in an N x M matrix structure. Constraint: Do not import NumPy or any external matrix library.

```
Sample Input:    Matrix = [[1, 2, 3], [4, 5, 6]]
Expected Output: Transpose: [[1, 4], [2, 5], [3, 6]]
```

4. Dictionaries & Sets

Utilizing key-value stores and unique hashable elements for associative data operations.

Q19. Alphanumeric Frequency Mapper

BASIC

Write a script that processes an input string sentence and returns a dictionary where keys represent unique characters and values capture their exact frequency count. Exclude empty space characters.

```
Sample Input:    Text = 'hello world'
Expected Output: Dictionary: {'h': 1, 'e': 1, 'l': 3, 'o': 2, 'w': 1, 'r': 1, 'd': 1}
```

Q20. Set Theory Intersection Challenge

BASIC

Given two lists of student database IDs, cast them into Python sets and extract: 1) IDs present in both databases, 2) IDs present in only the first database, and 3) All unique elements across both lists combined.

```
Sample Input:    ListA = [1, 2, 3, 4], ListB = [3, 4, 5, 6]
Expected Output: Both: {3, 4}, Only A: {1, 2}, Combined: {1, 2, 3, 4, 5, 6}
```

Q21. Dictionary Inversion & Grouping Registry

INTERMEDIATE

Write an algorithm to invert a dictionary's layout, transforming old values into new unique keys. If multiple source keys shared the exact same value, group those source keys together into a structured list under that inverted key value.

Sample Input: Data = {'Apple': 'Fruit', 'Carrot': 'Veg', 'Banana': 'Fruit'}
Expected Output: Inverted: {'Fruit': ['Apple', 'Banana'], 'Veg': ['Carrot']}

Q22. Multi-Collection Intersection Filter

INTERMEDIATE

Given three separate list arrays containing integers, construct an optimized Set-based filtration routine that extracts a list containing elements that appear in **at least two** out of the three input arrays.

Sample Input: L1 = [1, 2, 3], L2 = [2, 3, 4], L3 = [3, 4, 5]
Expected Output: Filtered Elements: [2, 3, 4]

Q23. Categorical Anagram Bucket Grouper

ADVANCED

Given an array collection of string words, design a dictionary grouping structure that buckets words together that are anagrams of one another. The keys of your dictionary should represent the normalized sorted character baseline.

Sample Input: Words = ['eat', 'tea', 'tan', 'ate', 'nat', 'bat']
Expected Output: Grouped: {'aet': ['eat', 'tea', 'ate'], 'ant': ['tan', 'nat'], 'abt': ['bat']}

Q24. Recursive Nested Dictionary Deep Merger

ADVANCED

Write a deep-merge function for Python dictionaries. Given dict1 and dict2, combine them recursively. If a key maps to another dictionary in both objects, merge those inner structures. If a key maps to a primitive value, dict2's value overrides dict1.

Sample Input: d1 = {'a': 1, 'b': {'x': 10}}, d2 = {'b': {'y': 20}, 'c': 3}
Expected Output: Merged: {'a': 1, 'b': {'x': 10, 'y': 20}, 'c': 3}

5. Conditional Statements (if-elif-else)

Constructing multi-path evaluation logic, testing complex boundary rules, and conditional state routing.

Q25. Rigorous Gregorian Leap Year Evaluator

BASIC

Build an absolute date-logic evaluator to check if a provided year is a leap year. A year is a leap year if it is divisible by 4, but not by 100, unless it is also perfectly divisible by 400.

```
Sample Input:   Year = 2000 | Year = 1900
Expected Output: True      | False
```

Q26. Dynamic Traffic Dispatcher

BASIC

Write an if-elif-else routing network that accepts a string representing a current light status ('Red', 'Yellow', 'Green') and a Boolean flag denoting an approaching Emergency Vehicle. Dispatch actions appropriately.

```
Sample Input:   Light = 'Red', Emergency = True
Expected Output: Action: 'Halt non-emergency lane. Override signal for Emergency
                  Vehicle.'
```

Q27. Geometric Triangle Validity & Classification

INTERMEDIATE

Accept three floating-point lengths representing triangle sides. First, confirm if they satisfy the Triangle Inequality Theorem ($a+b>c$). If valid, further classify the structure as Equilateral, Isosceles, or Scalene using conditional statements.

```
Sample Input:   Sides = 5.0, 5.0, 8.0
Expected Output: Valid Status: True | Classification: Isosceles Triangle
```

Q28. Progressive Income Tax Slab Engine

INTERMEDIATE

Develop a progressive income tax engine based on these tax tiers: Up to \$10,000 -> 0% Tax; next \$40,000 (\$10,001 to \$50,000) -> 10% Tax; next \$50,000 (\$50,001 to \$100,000) -> 20% Tax; Earnings above \$100,000 -> 30% Tax. Given an income, compute exact total tax due.

Sample Input: Income = \$65,000

Expected Output: Total Calculated Tax: \$7,000 (0 + 4000 + 3000)

Q29. Quadratic Complex Discriminant Roots Engine

ADVANCED

Write an algebraic root-solver for quadratic equations ($ax^2 + bx + c = 0$). Request coefficients a , b , and c . Using the mathematical discriminant ($D = b^2 - 4ac$), map distinct conditional pathways to handle and format two real roots, one repeated real root, or complex imaginary roots.

Sample Input: $a = 1, b = -5, c = 6$ | $a = 1, b = 2, c = 5$

Expected Output: Roots: 3.0 and 2.0 | Roots: -1+2j and -1-2j

Q30. Calendar Date Metric Validator

ADVANCED

Accept three individual integers representing Day, Month, and Year. Write a comprehensive conditional validation suite that checks if this combination forms a physically real calendar date. Factor in variable month lengths (28, 30, 31 days) and Leap Year exceptions for February.

Sample Input: Day = 29, Month = 2, Year = 2021

Expected Output: Date Validity: False (2021 is not a leap year)

6. Loops (for & while)

Implementing definite/indefinite execution loops, loop interruption flags, and nested spatial matrices.

Q31. Iterative Factorial Tracker

BASIC

Write a program using an indefinite while loop that computes the mathematical factorial ($N!$) of a given user input integer. Avoid any recursive function calls.

Sample Input: Number = 5

Expected Output: 5! Result: 120

Q32. Fibonacci Sequence Array Generator

BASIC

Write a script utilizing a definite for loop to calculate and display the first N sequence terms of the historical Fibonacci progression sequence (0, 1, 1, 2, 3, 5, 8, ...).

Sample Input: $N = 7$

Expected Output: Sequence Output: [0, 1, 1, 2, 3, 5, 8]

Q33. Prime Number Interval Extractor

INTERMEDIATE

Write a program utilizing nested loops that checks an interval span between integer start and integer end, and outputs a formatted list tracking every prime number within that range.

Sample Input: Start = 10, End = 30

Expected Output: Identified Primes: [11, 13, 17, 19, 23, 29]

Q34. Collatz Conjecture Traversal String

INTERMEDIATE

Design an indefinite while loop system tracking the Collatz Conjecture: Start with positive integer n . If n is even, divide it by 2; if odd, multiply it by 3 and add 1. Repeat this operational track until n reaches 1. Print out the full trail path sequence and total steps taken.

Sample Input: Initial $n = 6$

Expected Output: Path: 6 -> 3 -> 10 -> 5 -> 16 -> 8 -> 4 -> 2 -> 1 | Total Steps: 8

Q35. Nested Combinatorial Pascal Triangle Matrix

ADVANCED

Write a nested looping program that generates Pascal's Triangle down to an arbitrary N depth rows. Represent rows as lists nested within an encompassing list structure, mapping binomial coefficient sums.

Sample Input: Rows = 5

Expected Output: Triangle: `[[1], [1, 1], [1, 2, 1], [1, 3, 3, 1], [1, 4, 6, 4, 1]]`

Q36. Longest Consecutive Sequence Identifier

ADVANCED

Given an completely unsorted input list array of integers, construct a loop tracking routine that scans the list and determines the longest consecutive run length of numbers found anywhere inside the collection.

Sample Input: List = `[100, 4, 200, 1, 3, 2]`

Expected Output: Longest Consecutive Sequence Length: 4 (Sequence: `[1, 2, 3, 4]`)

7. Functions, Scope & Lambda Expressions

Designing modular routines, utilizing positional/keyword argument unpacking, working with lexical scope/closures, and implementing functional patterns.

Q37. Dynamic Invoice Total Calculator

BASIC

Write a function `calculate_invoice` that computes a billing total. It should accept a variable number of unique item prices via `*args`, and discount or tax metadata settings via `**kwargs`. Ensure it safely extracts optional arguments like `'discount_rate'` or `'tax_rate'` with robust fallback defaults.

Sample Input: `calculate_invoice(100, 50, 25, discount_rate=0.10, tax_rate=0.05)`

Expected Output: Final Total: 165.375

Q38. In-line Lambda Dictionary Sorting

BASIC

Given a list of dictionary elements representing products with 'name' and 'price' fields, use Python's built-in `sorted()` mapping engine bound alongside an inline custom anonymous `lambda` expression to sort the objects by price in ascending order.

```
Sample Input:    [{'name': 'A', 'price': 50}, {'name': 'B', 'price': 20}]
Expected Output: [{'name': 'B', 'price': 20}, {'name': 'A', 'price': 50}]
```

Q39. Recursive Arbitrary Multi-Nested List Flattener

INTERMEDIATE

Write a recursive function `flatten_list` that accepts a multi-dimensional array structure (lists containing integers or other nested lists at infinite depths) and reduces it down into a completely flat uniform single-dimensional list.

```
Sample Input:    flatten_list([1, [2, [3, 4], 5], 6, [7]])
Expected Output: Flattened List: [1, 2, 3, 4, 5, 6, 7]
```

Q40. Stateful Lexical Closure ID Generator

INTERMEDIATE

Design a function `create_id_generator` that demonstrates lexical scoping and variable encapsulation. It must return an inner function closure that generates sequential, prefixed identity tracking strings (e.g., 'EMP-1001', 'EMP-1002') every single time it is evaluated, without modifying any global state variables.

```
Sample Input:    gen = create_id_generator('EMP', start=1000); gen(); gen()
Expected Output: First call: 'EMP-1000' | Second call: 'EMP-1001'
```

Q41. Performance Execution Time Metrics Decorator

ADVANCED

Create a reusable function decorator named `@track_execution_time`. When added on top of any target computational function, it wraps execution logic, monitors precision duration benchmarks, and automatically logs performance diagnostic outputs.

```
Sample Input:    @track_execution_time
                  def compute_squares(): ...
Expected Output: Console Log: 'Function compute_squares executed in 0.0412 seconds.'
```

Q42. Custom Data Stream Sliding Window Generator

ADVANCED

Implement a memory-efficient generator function `sliding_window` using the `yield` statement. It must take an iterable sequence and an integer window width length, yielding consecutive sliding subset chunks from the stream without copying massive sequences in memory layout.

```
Sample Input:    list(sliding_window([1, 2, 3, 4, 5], size=3))
Expected Output: [[1, 2, 3], [2, 3, 4], [3, 4, 5]]
```

8. Object-Oriented Programming (OOPs)

Constructing reusable objects, applying access modifiers, executing multi-inheritance overrides, and overloading built-in protocols.

Q43. Encapsulated Library Asset Blueprint

BASIC

Create a `Book` class that models properties: `title`, `author`, and `is_checked_out`. Provide a clean custom string interpretation method `__str__` to output state gracefully, along with instance methods `checkout()` and `return_book()` that toggle state safely.

```
Sample Input:    b = Book('Python Core', 'Guido'); b.checkout(); print(b)
Expected Output: 'Python Core by Guido - Status: Checked Out'
```

Q44. Simulated Constructor Polymorphism Blueprint

BASIC

Python does not support traditional constructor overloading natively. Create a class `Employee` whose default `__init__` constructor maps basic details, but add an explicit `@classmethod` named `from_string()` that parses a raw hyphen-delimited text record to initialize and return a fully hydrated class object model.

```
Sample Input:    emp = Employee.from_string('John-Doe-DataEngineer')
Expected Output: emp.first_name = 'John', emp.role = 'DataEngineer'
```

Q45. Hierarchical Vehicle Super-Class Method Overrides

INTERMEDIATE

Design a base structural class `Vehicle` containing a method `describe()`. Derive a subclass `ElectricCar` that utilizes `super().\_\_init\_\_()` to inherit structural configurations from the parent class, while providing a specialized `describe()` function override that prints out internal battery metrics as well.

```
Sample Input:    ecar = ElectricCar('Tesla', 'Model 3', range=350); ecar.describe()
Expected Output: 'Tesla Model 3 with a 350-mile electric battery range profile.'
```

Q46. Secure Bank Vault Property Access Encapsulation

INTERMEDIATE

Design a `BankAccount` class using secure prefixing access modifiers to make account balance private (`\_\_balance`). Implement clean accessors utilizing the official `@property` and `@balance.setter` decorators. Enforce custom business rules (e.g., balance updates cannot be negative numbers).

```
Sample Input:    acc = BankAccount(100); acc.balance = -50
Expected Output: Raises: ValueError('Account balances cannot drop below absolute
                  zero.')
```

Q47. Abstract Factory Pattern for Payment Engines

ADVANCED

Utilize Python's native `abc` module to design an abstract contract blueprint class `PaymentProcessor` that contains an `@abstractmethod` definition named `process\_transaction()`. Create specialized concrete subclasses `StripeProcessor` and `PayPalProcessor` that conform to this interface to show polymorphic runtime execution.

```
Sample Input:    processors = [StripeProcessor(), PayPalProcessor()];
                  [p.process_transaction(50) for p in processors]
Expected Output: Polymorphic calls successfully dispatching individual target
                  implementations.
```

Q48. Mathematical Dunder Vector Operator Overloading

ADVANCED

Write a structural multi-dimensional class ``Vector``. Use special magic methods (dunder systems) to overload standard mathematical operations: ``__add__`` to perform standard element-wise coordinate additions, and ``__mul__`` to calculate dot products between independent `Vector` structures.

```
Sample Input:    v1 = Vector(2, 4), v2 = Vector(1, 3); v1 + v2 | v1 * v2
Expected Output: Vector(3, 7) | 14  (since 2*1 + 4*3 = 14)
```

9. Exception Handling

Trapping standard execution runtime errors, implementing recovery procedures, and establishing custom domain exceptions.

Q49. Robust Direct Numeric Cast Sanitization Engine

BASIC

Write an interactive calculation utility loop that reads dynamic text inputs from a terminal and casts them into integers. Implement a try-except layout blocking ``ValueError`` errors if the string contains alphabetic symbols, and blocking ``ZeroDivisionError`` errors when performing divisions.

```
Sample Input:    Input: 'abc' | Input: '0'
Expected Output: Handled: 'Invalid format numerical cast failed' | 'Division by zero
                  forbidden'
```

Q50. Secure Database Connection Resource Cleanup Guard

BASIC

Demonstrate the complete lifecycle of exception handling blocks (``try``, ``except``, ``else``, ``finally``). Write a function simulating a database entry transaction. If an error is triggered, catch it; if it completes successfully, log a success statement via ``else``; and always ensure system configurations are closed via ``finally`` execution.

```
Sample Input:    execute_transaction(valid_query=True)
Expected Output: 'Transaction logged successfully. Closing database connections
                  resources cleanly.'
```

Q51. Safe Nested Dictionary Path Traversal Handler

INTERMEDIATE

Write a deep dictionary key extractor utility function `safe_lookup`. It must navigate multiple nested dictionaries given a sequence path of tracking keys. Trap `KeyError` or `TypeError` anomalies safely within execution buffers and return a clean `None` default fallback without crashing.

Sample Input: `safe_lookup({'a': {'b': 2}}, ['a', 'x', 'y'])`

Expected Output: Return Value: None

Q52. Explosion Propagation Context Importer

INTERMEDIATE

Write a routine that demonstrates python exception chaining principles using the syntax `raise NewException from OldException`. Capture a low-level error (like `IndexError`) inside a custom sequence extraction routine and wrap it up inside a high-level context exception (like `SystemExecutionError`).

Sample Input: `extract_index_record(out_of_bounds_index=99)`

Expected Output: Console Stack Trace: 'SystemExecutionError caused by underlying
IndexError'

Q53. Custom Domain Warehouse Inventory Exception Suite

ADVANCED

Create a custom user-defined infrastructure exception class named `InventoryDeficitError` that inherits from Python's base `Exception`. Give it fields to track a product SKU code and total deficient quantity values. Write an inventory deduction routine that explicitly evaluates state rules and raises this error when stocks drop below demands.

Sample Input: `deduct_stock(sku='SKU-902', quantity_demanded=500)`

Expected Output: Raises: `InventoryDeficitError('SKU-902 requires 120 more units to fulfill total order.')`

Q54. Transactional Rollback Context Manager Class

ADVANCED

Design a custom context manager class named `TransactionalState` utilizing python dunder structures `__enter__` and `__exit__`. It must take an internal mutable state dictionary object. If any runtime exception is thrown inside the `with` code block container, `__exit__` should trap the error and roll back the dictionary state to its original configuration before the block started.

Sample Input: `with TransactionalState(my_dict): my_dict['balance'] = 5000; raise RuntimeError()`

Expected Output: State rolled back to pre-block conditions. Exception handled safely.

10. File Handling & I/O Systems

Managing non-volatile storage, streaming character datasets, parsing structured records, and enforcing serialization safety wrappers.

Q55. Text File Metadata Metric Parser

BASIC

Write a program that opens an existing plain text file input in read-only mode. It must iterate through the lines of text to calculate and print three unique operational metrics: total lines, total unique whitespace-separated words, and total standalone characters.

Sample Input: File content: 'Hello World\nWelcome to Python'

Expected Output: Metrics Summary: 2 lines, 5 words, 33 characters.

Q56. Alternating File Line Stream Merger

BASIC

Write a script that opens two distinct input files (`fileA.txt`, `fileB.txt`) simultaneously. It must read their contents and merge lines in an alternating zip sequence layout, streaming the compiled structural string output out into a third target log file (`combined.txt`).

Sample Input: A: Line1\nLine2 | B: Alpha\nBeta

Expected Output: combined.txt: Line1\nAlpha\nLine2\nBeta

Q57. Pure Procedural CSV Core Record Extractor

INTERMEDIATE

You are given a raw comma-separated values CSV log file. Constraint: Without importing Python's built-in `csv` module, read the spreadsheet, break up fields manually using string partitioning, skip the initial structural headers, and compile a subset list of rows filtered based on custom conditional values.

Sample Input: File tracking employees: 'Name, Age, Role\nAlice, 30, Dev\nBob, 19, Intern'

Expected Output: Filtered output for Age > 25: [['Alice', '30', 'Dev']]

Q58. Find-and-Replace In-Place Buffer Utility

INTERMEDIATE

Write an administrative disk tool utility that opens a target configuration file, loads its string segments safely into a localized memory stream buffer, searches for a target placeholder word keyword sequence, executes an automated find-and-replace routine, and saves the corrected payload back to disk.

Sample Input: Target keyword: 'DEV_HOST' -> Replace with: 'PROD_DB'

Expected Output: File updated and rewritten successfully with zero content duplication errors.

Q59. Transactional JSON Micro-Database Ledger Updates

ADVANCED

Implement an automated update engine for a local database serialized as a raw structured JSON file text structure. Use Python's built-in `json` module to load data safely into memory, dynamically search nested records to update balance logs or metadata keys, and securely write changes back to the disk file with clean indentation styling.

Sample Input: update_json_db(user_id=102, points_added=25)

Expected Output: Local JSON structure updated, validated, and serialized to disk format accurately.

Q60. Automated Byte-Size Boundary Log Rotator

ADVANCED

Write a robust monitoring function `append_system_log(message)` that appends timestamped text records to a system log file. If the targeted text file's current size on disk exceeds a fixed byte-size threshold capacity (e.g., 5KB), the tool must perform an automated backup file rollover routine, renaming the full file to `system.log.1` before opening a fresh blank ledger.

Sample Input: `append_system_log('Critical memory warning log flag triggered.')`

Expected Output: Log added. Rollover executed seamlessly; system.log.1 created.